Scythe: Semantic-Aware Rewriting for Fast, Effective Program Reduction

Ioanna Moraiti[1], Charalambos Mitropoulos[2], Artem Anchikov[3], Sotirios Ioannidis[2]
Dimitris Mitropoulos[4,5], Thodoris Sotiropoulos[1]
[1]Unaffiliated, [2]Technical University of Crete, [3]ETH Zurich,
[4]University of Athens, [5]National Infrastructures for Research and Technology
{ioannamoraiti@outlook.com, theosotr@windowslive.com}, {cmitropoulos, sioannidis}@tuc.gr, aanchikov@ethz.ch, dimitro@ba.uoa.gr

arXiv:2611.06515v1 [cs.SE] 4 Nov 2026

***Abstract*—Automated program reduction shrinks a program into a smaller one that preserves a property of interest. Language-agnostic grammar-based reducers are easy to retarget across languages, but because they reason only about syntax, they cannot perform the coupled edits needed to remove whole declarations, e.g., a method cannot be deleted while it is still called. As a result, they waste oracle calls on edits that cannot succeed and often leave irrelevant declarations behind. We present SCYTHE, a *fast*, *semantic-aware pre-processing* step that runs in front of any grammar-based reducer without modifying it. SCYTHE encodes the program as a typed declaration dependency graph and performs a set of semantic-aware operators that decouple declarations from their use sites, making them removable while keeping the program well-typed. Fine-grained token-level minimization is then left to the downstream reducer. On 73 benchmarks across C, Java, and Solidity, combining SCYTHE with PERSES never yields a larger result than PERSES alone, and a strictly smaller one on 70 of 73 cases. On average, the combination produces programs 55.6% smaller (up to 95.5%) and is 2.3× faster (up to 7.4×). SCYTHE also boosts LATRA, a reducer that, unlike the purely syntactic PERSES, also applies language-specific transformations: the combination produces programs 10.6% smaller (up to 61.8%) and is 2.4× faster on average. Video: https://youtu.be/uEq_YYU0zZw**

***Index Terms*—Program reduction, delta debugging, compiler testing, semantics**

I. INTRODUCTION

Automated program reduction is integral to debugging and bug deduplication in software systems that take programs as input, such as compilers, interpreters, and static analyzers. Given a program P that exhibits a property of interest, such as a compiler crash, a miscompilation, or a static-analysis report, a reducer produces a smaller program P' that still exhibits it, querying an *oracle* (a script that checks the property) on each candidate [1]. Existing reducers fall into two families. Language-specific reducers, such as C-Reduce [1], apply transformations tailored to one language (e.g., C). These transformations make reduction effective, but the tools are hard to retarget to other languages. Language-agnostic, grammar-based reducers [2], [3] instead delete syntax while guaranteeing that the result still parses, so they can be retargeted to a new language simply by supplying its grammar. However, this generality comes at a cost. Language-agnostic reducers can be slow, even on small programs, and often leave behind code that is irrelevant to the property of interest.

Example: Consider a Java program that crashes the Java compiler (`javac`, JDK 8.0.32); it spans 300 lines with 16 top-level types, 55 methods, and heavy use of inheritance, generics, and lambdas. Figure 1a shows the version produced by PERSES [2], a state-of-the-art language-agnostic reducer, after roughly five minutes. Several remaining declarations (e.g., class `Base`, field `active`, method `label`) are irrelevant to the bug, yet PERSES cannot remove them because they are still referenced elsewhere.

The problem: Programs targeted by reduction tools contain many declarations unrelated to the property of interest (here, most of the 16 types and 55 methods). The goal is to *identify and remove these irrelevant declarations, ideally eliminating large blocks of code quickly with few oracle queries*. However, doing so requires breaking the dependencies that tie each declaration to its use sites, edits that a language-agnostic reducer cannot perform.

Our tool: We bridge the gap between language-agnostic and language-specific reducers with SCYTHE, a *fast*, *semantic-aware pre-processing* step that runs before a language-agnostic reducer. It targets developers and testers of compilers, interpreters, and static analyzers, e.g., when reducing fuzzer-found bugs. SCYTHE removes whole classes, methods, and fields, applying a set of *general, semantic-aware decoupling operators* that rewrite a declaration's use sites. The declaration then becomes removable while the program remains well-typed. SCYTHE supports C, Java, and Solidity, and is easily extensible to new languages. On our running example, the SCYTHE + PERSES pipeline produces the smaller program of Figure 1b in about two minutes (2.1× speedup), with *43%* fewer tokens than PERSES alone. Across 73 benchmarks, SCYTHE never makes the output of PERSES larger, at least halves it in 50 cases, and speeds up reduction by up to 7.4×. On Java, SCYTHE alone, in under a minute on average, already beats PERSES on 23 of 34 benchmarks. Paired with LATRA [4], SCYTHE yields programs 10.6% smaller (up to 61.8%) and reduction 2.4× faster on average, all *without* modifying the underlying reducers.

II. CHALLENGES

We identify two challenges that any reducer must overcome to remove irrelevant declarations (e.g., class `Base`, Figure 1a).

```java
class Main {
  Widget active;
  static String label(short code) {return "tag"; }
  final boolean READY =
    new Widget("first").check(active, () -> {
      String text = "x"; return text; })
    || active.check(new Widget(Main.label((short) 54)),
        ↪ null);
}
interface Supplier<R> {R get(); }
class Base {Base(Boolean flag, boolean extra) {}}
class Widget extends Base {
  Widget(String tag) {super(false, true); }
  Boolean check(Widget other, Supplier task) {return
      ↪ true; }
}
```

(a)

```java
class Main {
  final boolean READY =
    ((__A) null).check(null, () -> {
      String text = "x"; return text;
    })
    || ((__A) null).check(null, null);
}
interface Supplier<R> {R get(); }
class __A {
  Boolean check(__A other, Supplier task) {return
      ↪ true; }
}
```

(b)

Fig. 1: An example of a Java program that crashes JDK 8. (a) The minimized program produced by PERSES, a language-agnostic reducer; (b) The minimized program produced by PERSES after the code is pre-processed by SCYTHE.

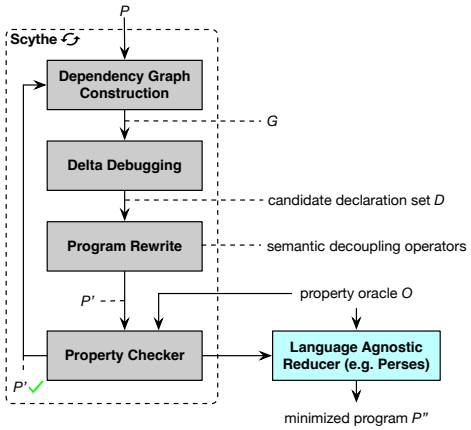


Fig. 2: The high-level architecture of SCYTHE.

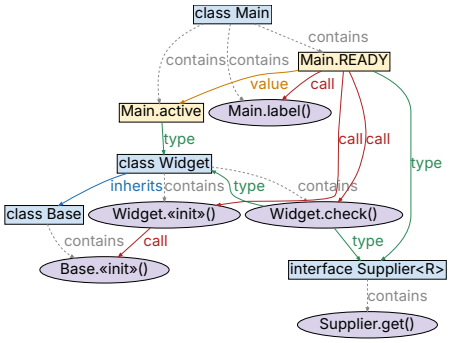


Fig. 3: The dependency graph of the program of Figure 1a.

Challenge 1: program dependencies that block removal: Pruning a declaration usually makes the oracle reject the program, even when the declaration is irrelevant to the bug, because it is referenced at multiple sites. For example, pruning class `Base` alone makes the crash disappear, since `Base` still appears after `extends` (line 11) and its constructor is called (line 12). A reducer must therefore identify every *dependency* between a declaration and its uses, and different uses demand different edits. Removing the method `label` requires dropping or replacing its call. Removing a class used as the type of a field, such as `Widget` for `active` (line 2), may require removing the field and, in turn, every use of that field.

Challenge 2: Syntax-guided search is prohibitively expensive: Existing reducers employ sophisticated search to find further reduction opportunities. VULCAN [3] guarantees tree-minimality, a stronger property than 1-minimality, through exhaustive, *syntax-guided* search that can discover that removing `Base` also requires removing its `super` call and the inheritance. However, this is very costly and still misses opportunities [4]: VULCAN cannot reduce our example even after six hours.

III. SCYTHE

Overview: Figure 2 shows the high-level architecture of SCYTHE, which runs as a fast pre-processing step *before* the actual language-agnostic reducer. Its goal is *not* to produce a fully minimized program or to remove every redundant token, but to quickly remove bulk code (*Challenge 2*) and whole declarations that the downstream reducer would struggle to reason about (*Challenge 1*). SCYTHE takes a source file P and a property oracle O, and outputs a program P' for the downstream reducer. The two may alternate until the program stops shrinking, as each can expose new opportunities for the other. Internally, SCYTHE iterates four steps until a fixpoint.

Step 1: dependency graph construction: SCYTHE parses the input program P into a graph that we call the *typed dependency graph*. Each node represents a declaration, such as a function, field, or class. Each edge records how one declaration uses another. The edge labels let SCYTHE pick the right way to break each dependency before removing a declaration (see *Step 3*). The graph (Figure 3) captures four kinds of relationships: (1) *containment* (`Main` contains `Main.active`), (2) *inheritance* (`Widget` inherits from `Base`), (3) *type* (`Main.active` has type `Widget`), and (4) *value/call* (`Main.READY` reads `Main.active`).

Step 2: delta debugging: SCYTHE applies delta debugging [5] over the nodes of G, i.e., at the level of declarations. Rather than a single reduction list over all nodes, SCYTHE schedules a sequence of passes, each on a single declaration type, ordered from coarse to fine-grained (e.g., classes, then

TABLE I: SCYTHE's semantic decoupling operators.

Declaration d (Kind)	**Use Site u (Kind)**	**Operator**
Class	`inherits`	REMOVE(u)
	`type`	ADDCLASS(u, `__A`); REPLACE(u, `__A`)
Function	`call` (result used)	REPLACE(u, DEFAULT($ret(d)$))
	`call` (unused result)	REMOVE(u)
Any	`value`	REPLACE(u, DEFAULT($type(d)$))
Any	`contains`	REMOVE(d)

methods, then fields). Deleting a class discards all its nested code in one oracle call. Each pass yields a candidate set $D \subseteq V(G)$, tested against the oracle (Step 4). The process guarantees 1-minimality at the level of declarations.

Step 3: program rewrite: For each declaration in the candidate set D, SCYTHE removes the declaration node and applies a set of *semantic-aware decoupling operators* that detach it from its use sites. To decide which operators to apply, SCYTHE examines the incoming edges of the node (its uses), together with those of the declarations it contains. The kind of reference selects the operator for each use (Table I).

For example, to remove class `Base` (Figure 1a, line 10), SCYTHE applies REMOVE to its `inherits` edge from `Widget`, dropping the `extends` clause, and to the call of its constructor from `Widget`'s constructor, whose result is unused. With both edges gone, `Base` and its constructor are deleted (Figure 1b). Similarly, a type use of a removed class is rewritten to a placeholder class `__A` (ADDCLASS; REPLACE). A value use, or a call whose result is used as an expression, is replaced by the default value of the corresponding type (DEFAULT($type(d)$)). This is why the references to `Widget` and the field `active` collapse to `((__A) null)` in Figure 1b (lines 3, 6).

Step 4: property checking: If the rewritten P' still satisfies the oracle O, the change is committed (feedback arrow in Figure 2); otherwise, it is rolled back. Faulty rewrites thus never reach the output, assuming, like any reducer, a deterministic oracle.

A. Implementation, Usage, and Availability

SCYTHE is implemented in ∼4,000 lines of Python. Parsing is delegated to `tree-sitter` [6]; adding a new language only requires mapping `tree-sitter` node types to SCYTHE's declaration and edge kinds and instantiating the operators of Table I. SCYTHE relies on the `picire` [7] library for delta debugging, but could be easily configured to use any list reduction algorithm (e.g., probabilistic delta debugging [8]). Users pass the input program, the oracle, and, optionally, a wrapper for the downstream reducer:

```
scythe --language java --source-file A.java \
  --script oracle.sh --post-processor perses.sh
```

SCYTHE is open source (GPL-3.0) at https://github.com/itiarom/scythe, with a Docker image and usage instructions; the artifact is at https://doi.org/10.5281/zenodo.23589196.

TABLE II: Sample results: size in tokens (*Tok*), time in seconds (*s*), and bug type (*Prop*): *U* (unexpected error), *CR* (crash), *M* (miscompilation).

			S		P		S+P	
Benchmark	Prop	Init	Tok	*s*	Tok	*s*	Tok	*s*
C								
`clang-31259`	M	48,079	2,101	744	518	4,111	240	3,546
`gcc-64990`	M	147,002	1,861	1,807	435	4,624	104	3,687
`gcc-61917`	CR	84,473	2,471	2,515	579	5,956	125	4,177
`gcc-59903`	CR	56,951	2,635	1,839	722	5,171	302	4,723
Java								
`jdk11-10`	U	5,298	83	98	657	731	31	124
`jdk11-7`	U	3,158	80	55	382	343	31	81
`jdk8-24`	CR	4,691	131	73	608	796	74	121
`jdk8-34`	CR	3,377	143	44	591	649	95	116
Solidity								
`smart17`	U	2,570	69	65	93	298	16	86
`smart7`	U	4,103	143	60	92	325	33	133
`smart13`	U	1,500	55	48	84	161	38	91
`smart4`	U	1,288	100	22	64	122	29	59

IV. PRELIMINARY EVALUATION

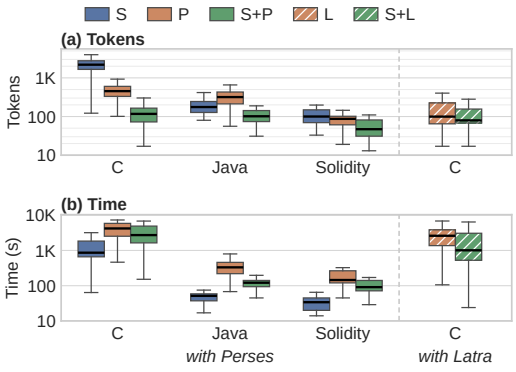
We evaluate whether SCYTHE improves the (**RQ1**) quality and (**RQ2**) efficiency of language-agnostic reducers.

Benchmarks: No public dataset spans C, Java, and Solidity, so we collect 73 programs from three sources. *C (20 benchmarks)*: the PERSES benchmarks [2], Csmith [9]-generated programs that trigger crashes and miscompilations in GCC and LLVM, widely used in prior work [2], [4], [10]. *Java (34 benchmarks)*: we generate programs with HEPHAESTUS [11] against OpenJDK: 20 programs that `javac` (JDK 11) rejects with an unexpected compile-time error and 14 that crash `javac` (JDK 8). *Solidity (19 benchmarks)*: smart contracts from the dataset of Mitropoulos et al. [12], each triggering a different warning or error in the Slither static analyzer [13].

Reducers: We pair SCYTHE with two state-of-the-art reducers: the grammar-based PERSES [2] and LATRA [4], which adds language-specific transformation templates and performs comparably to C-Reduce. We run LATRA only on C, as it does not support Java or Solidity. We exclude C-Reduce [1]: it is C-specific, whereas SCYTHE is designed to *boost language-agnostic reducers, so the two are complementary rather than competing*. For each benchmark and reducer R, we run three pipelines: (1) SCYTHE alone (**S**), (2) R alone (**P** or **L**), and (3) SCYTHE followed by R (**S+P** or **S+L**).

Results: Table II shows four benchmarks per language (all are in our artifact), and Figure 4 shows the distributions across all benchmarks. SCYTHE's own runtime is small (14 minutes, 50 seconds, and 30 seconds on C, Java, and Solidity). **S+P** substantially reduces final program size by 71.7%, 59.5%, and 31.6% on average for C, Java, and Solidity, respectively. **S+P** accelerates reduction by 1.6×, 3×, and 1.6× on average for C, Java, and Solidity, respectively. On C, this saves 23 minutes per benchmark on average and up to 2.5 hours. Overall, **S+P** is never larger than **P**, strictly smaller on 70 of 73 benchmarks, and faster on 67.

Beyond PERSES: The benefits carry over to LATRA: **S+L** is 2.4× faster than **L** on average (up to 7.3×), cutting total time from 19.0 to 11.4 hours. Although LATRA already applies



	with Perses			with Latra
S+R vs. R	C	Java	Solidity	C
Δ% tokens, mean	−71.7%	−59.5%	−31.6%	−10.6%
Δ% tokens, median	−74.0%	−61.2%	−25.4%	−15.2%
Speedup, mean	1.6×	3.0×	1.6×	2.4×
Speedup, median	1.3×	2.4×	1.6×	2.0×

Fig. 4: Reduction size (a) and time (b) of SCYTHE alone (S), a reducer R alone (P: PERSES; L: LATRA, hatched), and SCYTHE followed by R (S+R).

language-specific transformations, **S+L** yields smaller programs than **L** in 13 cases and equal ones in one (15.2% smaller at the median, up to 61.8%). In six cases, the output grows by 3–86 tokens, as SCYTHE's default values (Table I), e.g., `0xDEADBEEF` for an `int`, can hinder LATRA's transformations; we plan to fix this in future work (Section VI).

V. RELATED WORK

The closest line of work is grammar-guided reduction over a syntactically valid parse tree. PERSES [2] introduced the quotient representation that keeps every candidate parseable, PARDIS [14] prioritises subtrees by expected gain, and type-batched reduction [15] amortises oracle cost across simultaneous queries. All operate *within* a single parse tree, whereas SCYTHE runs one step *before* them, pruning whole declarations on a graph abstraction, and composes with any of them.

The remaining reducers fall into two groups. Delta-debugging engines (`ddmin` [5], HDD [16], PICIRE [7], PROBDD [8]) improve the underlying search policy and are orthogonal to SCYTHE: any can replace its delta-debugging core. Language-specific reducers (C-REDUCE [1], GTR [17], LPR [18]) encode per-language passes (hand-coded, learned, or LLM-suggested) and reach smaller results, but work in only one language. SCYTHE complements both families, and even improves LATRA, which performs on par with C-Reduce.

VI. CONCLUSION AND FUTURE WORK

We presented SCYTHE, a semantic-aware front-end that makes language-agnostic reducers both faster and more effective, without modifying them. Paired with PERSES, SCYTHE never produced a larger result across 73 benchmarks in three languages, halved the output in 50 of them, and cut total reduction time from 31.2 to 21.0 hours, lowering the compute and energy cost of debugging. Beyond this preliminary evaluation, we plan to select operators (Table I) and default values context-sensitively, to open further opportunities downstream and avoid the LATRA regressions.

REFERENCES

[1] J. Regehr, Y. Chen, P. Cuoq, E. Eide, C. Ellison, and X. Yang, "Test-case reduction for C compiler bugs," in *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2012, pp. 335–346.

[2] C. Sun, Y. Li, Q. Zhang, T. Gu, and Z. Su, "Perses: Syntax-guided program reduction," in *Proceedings of the 40th International Conference on Software Engineering*. ACM, 2018, pp. 361–371.

[3] H. Xu, Z. Wang, S. Hao, J. Li, W. Wang, and L. Zhang, "Pushing the Limit of 1-Minimality of Language-Agnostic Program Reduction," in *Proceedings of the ACM on Programming Languages, OOPSLA*. ACM, 2023.

[4] Z. Xu, Y. Wang, Y. Tian, M. Zhang, and C. Sun, "Latra: A template-based language-agnostic transformation framework for effective program reduction," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2025, pp. 2274–2285.

[5] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.

[6] "Introduction to Tree-Sitter," https://tree-sitter.github.io/tree-sitter/, 2026, accessed online; June 24, 2026.

[7] R. Hodován and A. Kiss, "Practical improvements to the minimizing delta debugging algorithm," in *Proceedings of the 11th International Joint Conference on Software Technologies (ICSOFT)*. SciTePress, 2016, pp. 241–248.

[8] G. Wang, R. Shen, J. Chen, Y. Xiong, and L. Zhang, "Probabilistic delta debugging," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, 2021, pp. 881–892.

[9] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2011, pp. 283–294.

[10] Z. Xu, Y. Tian, M. Zhang, J. Zhang, P. Liu, Y. Jiang, and C. Sun, "T-Rec: Fine-grained language-agnostic program reduction guided by lexical syntax," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, Jan. 2025.

[11] Hephaestus Compiler Project, "Hephaestus: A compiler testing framework," https://github.com/hephaestus-compiler-project/hephaestus, accessed: 2026-06-12.

[12] C. Mitropoulos, M. Kechagia, C. Maschas, S. Ioannidis, F. Sarro, and D. Mitropoulos, "Broken agreement: The evolution of Solidity error handling," in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. ACM, 2024, pp. 257–268.

[13] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.

[14] G. Gharachorlu and N. Sumner, "Pardis: Priority aware test case reduction," in *Proceedings of the 22nd International Conference on Fundamental Approaches to Software Engineering (FASE)*. Springer, 2019, pp. 409–426.

[15] ——, "Type batched program reduction," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2023, pp. 727–739.

[16] G. Misherghi and Z. Su, "HDD: Hierarchical delta debugging," in *Proceedings of the 28th International Conference on Software Engineering*. ACM, 2006, pp. 142–151.

[17] S. Herfert, J. Patra, and M. Pradel, "Automatically reducing tree-structured test inputs," in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2017, pp. 861–871.

[18] M. Zhang, Y. Tian, Z. Xu, Y. Dong, S. H. Tan, and C. Sun, "LPR: Large language models-aided program reduction," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2024, pp. 261–273.